Executive Summary

The penetration testing cases below check the application's security against the most common web security threats. The testing focused on key security areas, including authentication, authorization, input validation, session management, and security configuration.

The application addresses almost all the bullet points on the security checklist provided in the project description. Authentication functions enforce password complexity, implement account lockout (to prevent brute-force and credential-stuffing attacks), and allow password resets only to people who have access to the email provided (email authentication). The application also enforces authorization control using decorator functions, preventing unauthorized access to documents and admin controls. To prevent data inference, the app returns generic errors (e.g., HTTP 404) indicating that the application could not find the item, to avoid leaking sensitive information about the existence of resources, such as documents.

Input validation mechanisms effectively prevent common injection attacks such as XSS, command injection, and path traversal. Any files uploaded to the application go through basic malware scanning (checking for common signatures / magic bytes), file type limiting and validation (to detect mismatches), size restrictions, etc. Session management is also handled securely with random session token generation (hashed in storage) and saved with proper cookie flags (HttpOnly, SameSite). Sessions are invalidated on logout, password reset, or simply over time due to expiration.

A few vulnerabilities were found during testing (see the findings section for more details). Because the application is deployed with Flask, HTTPS is not fully enforced. Additionally, sessions are not bound to IP addresses (due to the possibility of mobile users switching between cellular and WiFi), which could allow for session hijacking (with the right user-agent and a stolen session token). A stolen session token would only be possible if the person had physical access to the browser, as sensitive data at rest is encrypted/hashed. File scanning relies on very basic validation and may not detect advanced malicious payloads that were not directly accounted for.

Overall, the application is well-secured against the common web vulnerabilities. With the addition of enforced HTTPS (assuming the application is properly deployed), enhanced storage

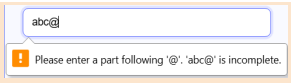
(through encrypted databases), and an additional file scanning system, the application would be suitable for deployment in a production environment.

Testing Methodology

Testing was performed using manual testing techniques, browser developer tools (inspect element), and crafted payloads to simulate common web app attacks.

Authentication Testing

- brute force protection
- password complexity enforcement
- session management
- logout functionality
- password reset security

test	payload/action	expected result	actual result
registration with valid inputs	username: testuser0 email: test0@example.com password: StrongPass123!	accepts valid inputs and creates a new user account	user successfully registered & stored in users.json, HTTP 201
registration with an invalid email	abc@ user@@test.com test@.com @test.com	rejects invalid email format	 browser blocked via HTML5 validation. Request not sent to app
registration with an invalid username	#$%^&* user name aaaaaaaaaaaaaaaaaaaa (21 chars)	app rejects usernames that don't match requirements	HTTP 400: "Invalid username"
registration with a short password	Abc123!	does not match the strong password policy, so it fails	HTTP 400: "Password does not meet requirements"
registration with a	strongpass123!	does not match the	HTTP 400:

valid length password but missing lower, upper, number, or symbol	STRONGPASS123! StrongPassword! StrongPass123	strong password policy, so it fails	"Password does not meet requirements"
successful login	testuser0 StrongPass123!	successful session creation	HTTP 200: redirect to dashboard
logout	press logout button on the dashboard	session successfully destroyed	HTTP 200: session successfully destroyed
fail once (invalid user or password or blank), then valid login	username: testuser0 password: wrongpass and then testuser0 StrongPass123!	error message for failed attempt, successful session created for valid credentials	HTTP 401: invalid username or password HTTP 200: redirect to dashboard & session created
account lockout	username: testuser0 password: wrongpass 5 failed logins on the same account (can be across different IPs)	temporarily lock the account after repeated failed attempts	HTTP 401: "account is temporarily locked" → authentication denied, can't log in for a period of time
brute force / rate limiting	username: testuser1 password: wrongpass 10 failed logins from the same IP in one minute (can be across different accounts)	limits repeated login attempts to prevent brute-force attacks	HTTP 429: blocked further requests temporarily
reset password with valid token	token: (real token from email) new password: NewStrong123!	successfully changed password, reflected in users.json as well (can no longer use old password)	HTTP 200: successfully changed password, cannot use old password, new password successful
reset password with invalid token	token: fake123 link will be: https://127.0.0.1:5000/reset-password-page?token=fake123	rejects invalid/expired reset tokens will let you attempt to set a password, but when you press reset	HTTP 400: "invalid token"

		password button, will print error	
reset password with used token	request email/link and try using twice	rejects used tokens will let you attempt to set a password, but when you press reset password button, will print error	HTTP 400: “token already used”

Authorization Testing

- horizontal privilege escalation
- vertical privilege escalation
- direct object reference
- forced browsing

Given one guest `guest1`, three users `testuser1`, `testuser2`, `testuser3` and one admin `admin`

testuser1 is owner of docA

testuser2 does not have any permissions for docA

testuser3 is editor of docA

guest1 is viewer of docA

Note: the app intentionally uses different codes depending on context. For sensitive things like documents, the app returns HTTP 404 (not found) rather than HTTP 403 (forbidden) to prevent revealing that a doc exists. For general role-based privilege escalation attempts (like a user or guest trying to access things above their level), the app will return HTTP 403 (forbidden) to show insufficient privileges.

test	payload/action	expected result	actual result
read another user’s	login as: testuser2	error: document not	HTTP 404:

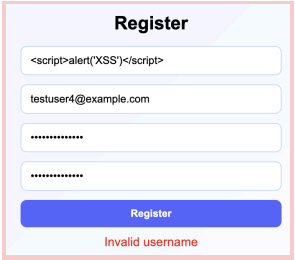
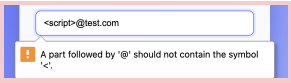
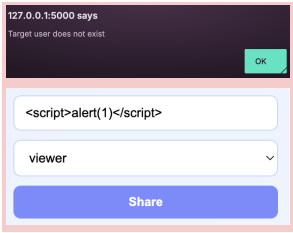
document without permission	download docA with 127.0.0.1:5000/download/<docA_doc_id>	found (even if docid exists, will say not found so it does not reveal info)	Pretty-print {"error":"Document not found"}
read another user's document with permission	login as: testuser3 attempt to download docA	download file	HTTP 200: downloaded file
delete or share/unshare another user's doc (as an editor)	login as: testuser3 attempt to delete / share docA	no authorization	HTTP 403: forbidden
guest tries to upload/share/unshare/delete	login as: guest1 try to upload, share, or delete docA	no authorization	HTTP 403: forbidden
non-admin tries admin pages or routes	login as: testuser1 try to access 127.0.0.1:5000/admin/dashboard OR 127.0.0.1:5000/admin/logs	no authorization	HTTP 403: forbidden
after admin demotes user, even if now-guest was editor on doc, cannot edit. only view	demote testuser3	should not be able to replace document now	HTTP 403: forbidden even if you promote testuser3 again, they are still viewer for that document (do not get permissions back)
private/incognito with no session tries to access anything /dashboard etc	in incognito, try: 127.0.0.1:5000/dashboard 127.0.0.1:5000/documents 127.0.0.1:5000/admin/dashboard	fail because no authentication	HTTP 401: authentication required
attempt to download, share, or delete	try: 127.0.0.1:5000/downl	fail, do not reveal existence of doc	error: "document not found"

guessed or modified doc_id	oad/<docA_doc_id>		

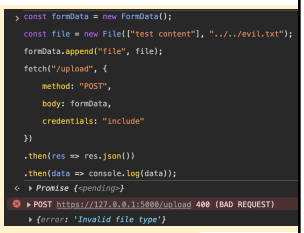
Input Validation Testing

(there will be a few overlapping categories)

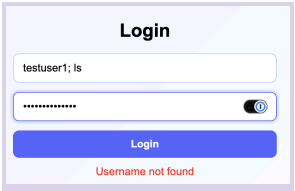
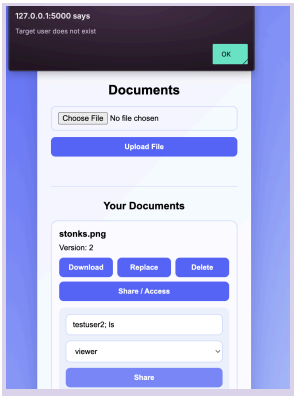
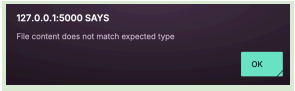
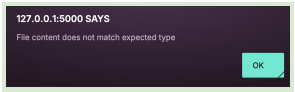
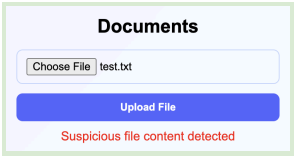
- XSS attempts
- path traversal
- command injection
- file upload vulnerabilities
- input length

test	payload/action	expected result	actual result
store XSS in username	register with username <script>alert('XSS')</script>	does not get stored	 "Invalid username"
store XSS in email	register with email <script>@test.com	does not get stored	 HTML5 catches the invalid email
XSS in the share target field	submit share input with target_username=<script>alert(1)</script>	does not execute	
XSS in filename	console input: const formData = new FormData(); const file = new File(["test"], '<script>alert("XSS")</scr	invalid input	HTTP 400: bad request 'invalid file type'

	ipt>.txt');
	
	formData.append("file", file);
	
	fetch("/upload", {
	method: "POST",
	body: formData,
	credentials: "include"
	})
	.then(res => res.json())
	.then(data => console.log(data));		

path traversal in filename upload	console input:
	const formData = new FormData();
	
	const file = new File(["test content"], "../../evil.txt");
	
	formData.append("file", file);
	
	fetch("/upload", {
	
	method: "POST",
	
	body: formData,
	
	credentials: "include"
	
	})
	
	.then(res => res.json())
	
	.then(data => console.log(data));
		invalid file type	
			HTTP 400: bad request
			
			‘invalid file type’

path traversal in download req	req /download/../../../app.py	url should not be found	**Not Found** The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.

command injection in username	test; ls OR test && whoami test | cat /etc/passwd	input treated as plain text… no command execution	
command injection in email/reset/share target field	submit test@example.com; cat /etc/passwd in login/reset/share field	input treated as plain text… no command execution	
oversized username/password	username: aaaaaaaaaaaaaaaaaaaa (21+ chars) password: aaaaaaaaaa... (130+ chars)	invalid username	invalid username
oversized file upload	upload file larger than 5mb	file too large	
file upload invalid extension	file.exe file.php file.js	invalid file type	
file upload invalid MIME type	Upload .pdf renamed to .jpg	file content doesn't match expected type	
file sig mismatch	rename image.png → file.pdf and upload	doesn't match expected type	
suspicious file content	test.txt included in /tests folder	suspicious file content detected	

Session Security Testing

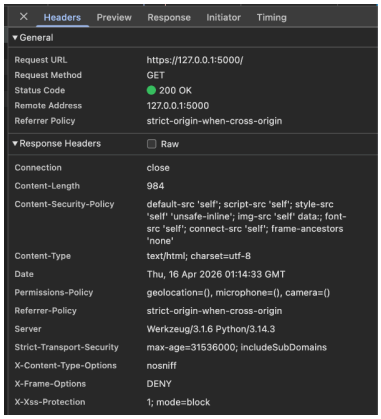
- session fixation
- session hijacking
- concurrent session handling
- session timeout

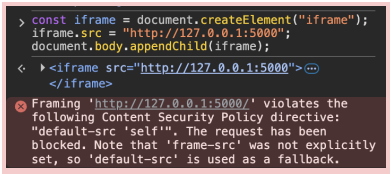
test	payload/action	expected result	actual result
attempt to set token manually (fixation)	manually set cookie: session_token=abc123 and then login	new session token should be generated (random) ^^ make sure token changes after logging in	random new token generated
session hijacking attempt	use stolen session token from another user copy session_token from cookie in browser A paste into browser B	access should fail unless attacker matches user agent… also theoretically no way to get session_token unless they had direct access to the user's browser	access fails if different user-agent is used
session reuse after logout	log in, copy cookie/token, log out, reuse old cookie/token	invalid token	invalid token
invalid session token	try using session_token=asdf	invalid token	invalid token
expired session	log in, wait 30 minutes, then try to access a doc or download or anything	says authentication is required	authentication is required
session persistence	log in, refresh page or revisit page before timeout	session remains valid	can see page
same user, two browsers	login in Chrome login in OperaGX	should be able to log in on both	works on both

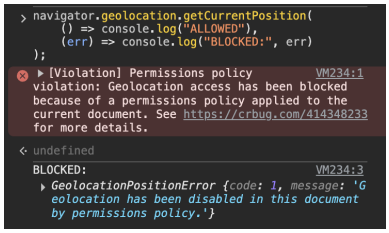
session binding	log in, copy cookie/token to a different browser/user agent	should not work because user-agent is different	session invalid
session binding (IP change)	log in, simulate IP change (if possible)	should still work	I couldn't test because i can't test localhost on my phone… not sure if grader can do this

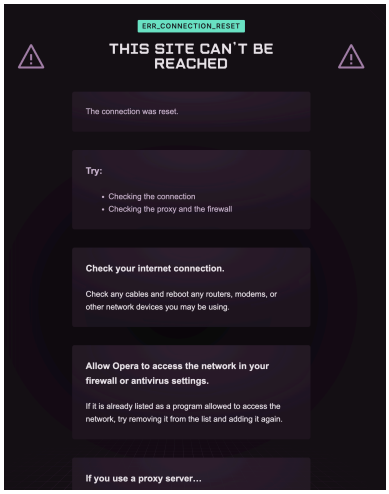
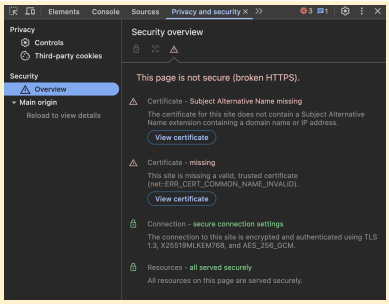
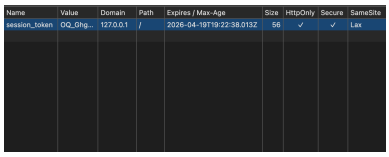
Configuration Testing

- security headers
- TLS/SSL configuration
- error handling
- debug mode disabled

test	payload/action	expected result	actual result
security headers	inspect response headers in browser check for: Content-Security-Policy X-Frame-Options X-Content-Type-Options Strict-Transport-Security Referrer-Policy Permissions-Policy	all security headers should be present	
content security policy	try inputting the following into any text boxes: <script>alert('CSP test')</script>	this will fail due to input validation, but even if it somehow got through, it would not execute	blocked by input validation

clickjacking protection	go to inspect element and to console
	
	paste in:
	const iframe = document.createElement ("iframe");
	iframe.src = "http://127.0.0.1:5000";
	document.body.appendChild(iframe);
		app cannot be embedded in an iframe	

MIME sniffing protection	input any invalid MIME types or mismatches	should not allow for invalid MIME types	X-Content-Type-Options: nosniff header was present. When testing with mismatched extensions, browser did not allow them to be uploaded
referrer policy	login as testuser1 so it redirects to dashboard into console, input: window.open("https://example.com");	referrer info should be limited when navigating cross-origin	 success: doesn't show rest of URL including documents or any other links
permissions policy	console: navigator.geolocation.getCurrentPosition(() => console.log("ALLOWED"), (err) => console.log("BLOCKED:", err));	unneeded browser features are disabled	
HSTS enabled	http://127.0.0.1:5000	browser is told to use HTTPS for future requests	not enforced due to lack of HTTPS for local Flask deployment header is present though!!!

			 Strict-Transport-Security max-age=31536000; includeSubDomains
HTTPS redirect	http://127.0.0.1:5000	should automatically connect to https://127.0.0.1:5000	no redirect occurred due to lack of TLS configuration (because we are using Flask for a local deployment)
TLS/SSL enabled	check inspect element → security tab to see that TLS is enabled	TLS is active, and certificate is presented (even though its self signed)	 TLS enabled, but HTTPS is broken
secure cookie config	inspect → application tab check cookie to see verify HttpOnly is true Secure is true SameSite is lax or strict	should use secure attributes	
debug mode disabled	/download/asdf ^^ should trigger error	should NOT show Flask	HTTP 404:

	because invalid doc id	stack trace	Pretty-print {"error":"Document not found"}
invalid input error handling	login with no input fields download invalid doc id upload no file ../../../../etc/passwd <script>fetch('http://attacker.com')</script>	clean JSON error no crash	no crash

Findings (Low, Medium, High, Critical)

Sessions not bound to IP address (Low)

Description

Session validation doesn't enforce IP address binding. It's only validated through user-agent binding and the token.

Impact

If a session token is somehow stolen, it could be reused from a different IP address (as long as the attacker manages to match the user-agent). This increases the risk of session hijacking.

Steps to Reproduce

Log in to the app and note the session_token in the cookie. Attempt to reuse the token in another browser session with the matching user-agent, and access will still be granted.

Proof of Concept

Testing demonstrated that session reuse is possible when user-agent conditions are met (I tried on two different browsers, but on the same user-agent). While I can only test from the same IP currently (due to local deployment), this could be a problem in production environments.

Remediation

Bind sessions to IP addresses if possible, though this may restrict certain mobile users due to cellular data and wifi switching. It's better to have them log in again than to have the site

be vulnerable to session hijacking. Perhaps I could also implement anomaly detection (suspicious behavior).

HTTPS not enforced (Medium)

Description

The app is configured to run over HTTPS using a self-signed certificate (that the user will generate) in the local deployment. As a result, HTTP connections (http://127.0.0.1:5000) fail to connect rather than redirecting to HTTPS, showing that the intended redirection logic is not functioning properly despite being implemented in the code.

Impact

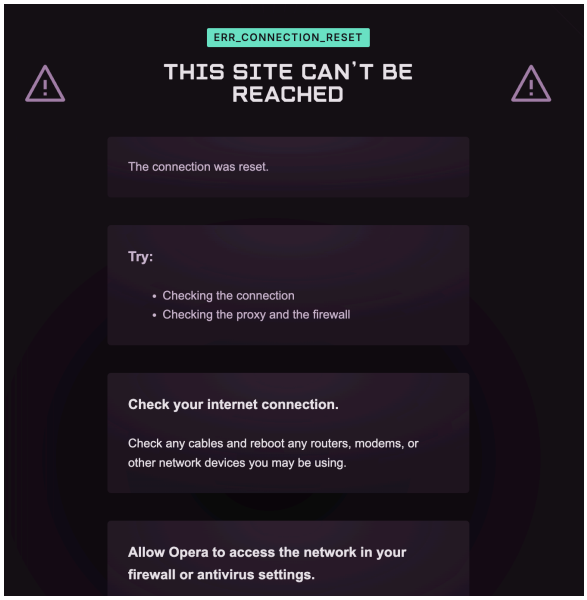
While HTTPS in enforced in practice, the lack of proper HTTP to HTTPS redirection could lead to inconsistent behavior in different deployment environments. In a production setting, if the person deploying the server configures the app to be run over HTTP, I don't believe they'll be able to use the app at all (since it would be running on HTTP, not HTTPS, and it would try to redirect to HTTPS).

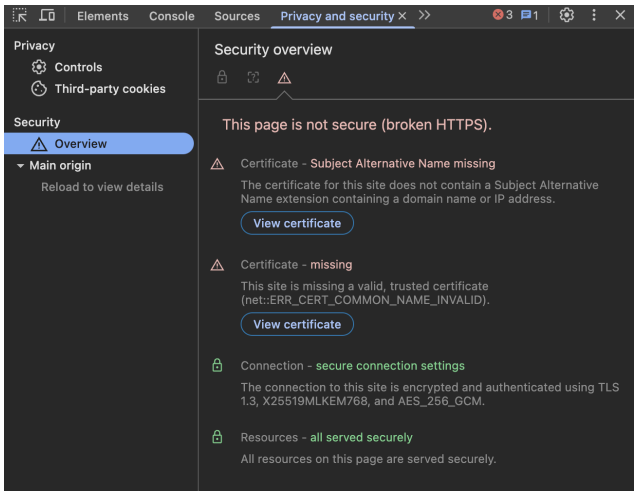
Steps to Reproduce

Go to http://127.0.0.1:5000 and you will see that the browser fails to connect rather than just redirecting to HTTPS. If you go to https://127.0.0.1:5000 instead, you will see that you can access the application.

Proof of Concept

The browser error page below is displayed when trying to access the app through HTTP, showing that the server doesn't successfully redirect them to HTTP.





Remediation

The app should be deployed on a proper production web server with proper TLS configuration. Make sure that all HTTP traffic is explicitly redirected to HTTPS using either server-level redirection or app-level redirection.

Limited Malware Detection (High)

Description

The app performs basic file validation by checking the filename, file extensions, MIME types, and magic bytes. It also does a very simple check to see if there is any scripting or PHP. However, it doesn't do any kind of advanced malware detection or antivirus scanning.

Impact

Malicious files that bypass simple checks (perhaps obfuscated code or advanced payloads) could be uploaded and stored in the system. If the code is accessed/executed by users, it could lead to some kind of exploitation.

Steps to Reproduce

Modify a malicious file to match allowed extensions and basic file signatures. Upload the file to the app. If it bypasses the validation, it will be stored on the system.

Proof of Concept

While I don't know enough about malicious scripting to provide an example, I know that there are definitely more malicious payloads than just files containing <script> and <?php>. As long as the file extension matches the magic bytes/file signature and the MIME type, and has a valid name (something simple like hello.txt), files containing the malicious code will be encrypted and stored.

Remediation

For the context of this project, I think the current code is sufficient. However, if it were to be used on a larger scale, this would be a security concern. I believe this could be fixed through integrating antivirus scanning of some sort or including a common malware signature check. Because the malware scanners are specifically designed to include all the common malware and are continuously updated, it will likely be more suited for scanning files (rather than trying to account for every individual threat myself).

Poor Key Management (Critical)

Description

The app currently stores the Fernet encryption key (data/secret.key) locally within the application directory. The key is used to encrypt/decrypt sensitive data like the uploaded documents.

Impact

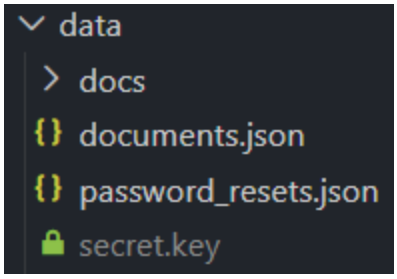
If an attacker somehow gains access to the server's file system, they will be able to use the key to decrypt all stored documents and sensitive data. This would be a complete loss in confidentiality since all encrypted data would be compromised.

Steps to Reproduce

Go to the app directory and find the key in data/secret.key. From there, you would be able to use the key with Fernet to decrypt the stored files (since documents are stored in data/docs (same parent folder).

Proof of Concept

As shown in the screenshot below, the secret.key file is in the data folder. The key is stored alongside the documents.



Remediation

Ideally, any encryption keys should be stored securely on a separate storage system or a key management system of some sort. Additionally, keys should be rotated periodically to ensure safe encryption-at-rest. If someone gains access to the key, they can decrypt all documents and recover user emails, which would be a critical security breach. Rotating periodically makes it so that if they recover a previous key (not the current one), they won't be able to decrypt the documents.

Conclusion

The penetration testing process demonstrated that the application is well-protected against a wide range of common web app vulnerabilities. It covers input validation, secure sessions, and authorization/authentication enforcement. The application prevents most injection attacks, brute force attempts, unauthorized access, and common file-based exploits (magic bits, type mismatch, scripts, binary, etc.).

However, due to the nature of the project, the application is locally deployed through Flask, preventing the HTTPS redirects and HSTS header from functioning as intended. Additionally, the key is generated locally and kept in the storage. Ideally, it should be encrypted and stored securely in a separate location. These limitations could be mitigated through secure server deployment, a key management system, and enhanced file scanning software. Overall, the application provides a solid security foundation and successfully mitigates the majority of tested attacks.